TransferIa · Documentos de cambios

Caras inclinadas y boca abajo

Entrega 3 · Detección con giros, máscara alineada y corrección de índices

Commit: `1b3f339` "mejorando angulos"

Fecha: 23 de octubre de 2026

Documento generado: 27 de octubre de 2026

Pruebas: caras generadas por IA, en CPU

Contenido

1. Resumen de la entrega 2
2. El problema 2
3. La investigación: dónde está la falla 3
4. La solución 4
4.1 Detectar probando giros 4
4.2 Enderezar las caras inclinadas 4
4.3 La máscara de oclusión alineada 5
4.4 La corrección de los índices 5
5. Cómo se comprobó 6
6. Cómo lo pruebas tú 6
7. Lo que aprendimos 7
Glosario 7

Si el índice aparece vacío, en Word haz clic derecho sobre él → Actualizar campos.

1. Resumen de la entrega

El commit `1b3f339` "mejorando angulos" (23 de octubre) hace que TransferIa funcione con **caras inclinadas, de costado y boca abajo**, y de paso corrige un error con los índices de caras.

Cambio	Qué mejora para el usuario	Archivo
Detección con giros	Encuentra caras giradas 90°, 180° o 270° si no las ve derechas	`core/Face_Analyzer.py`
Refinado de caras inclinadas	Los puntos de ojos, nariz y boca caen en su lugar aunque la cara esté inclinada	`core/Face_Analyzer.py`
Máscara de oclusión alineada	La máscara sigue la forma de la cara aunque esté girada	`core/xseg.py`
Corrección de índices	Ya no falla con `IndexError` al combinar ciertos índices	`core/Face_Swaper.py`

2. El problema

Con la versión anterior, si la persona estaba acostada, de cabeza o con la cabeza muy inclinada, el resultado salía **deformado**: ojos sobre la frente, boca en la nariz, o una mezcla borrosa.

Cómo reproducirlo

1. Toma una foto de destino con una cara derecha y gírala 90° y 180° (aquí, una cara generada por IA).
2. Cambia la cara con la versión `5351e26` y oclusión natural.
3. Compara: derecha sale bien; a 90° y 180° sale mal.

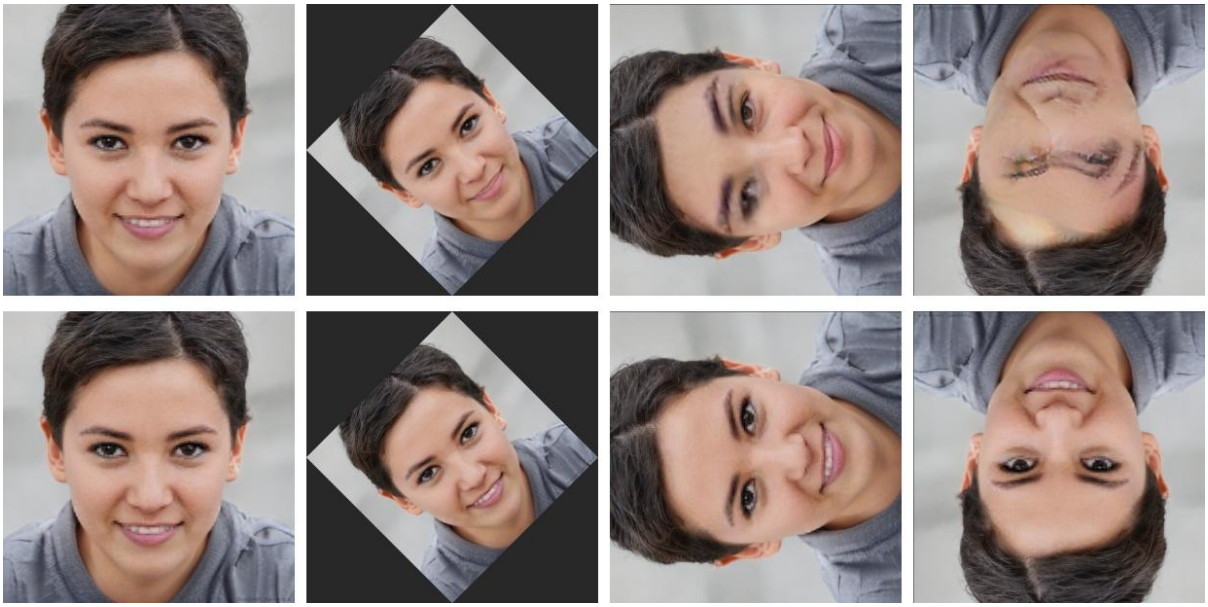


Figura 1. *Arriba, versión anterior; abajo, versión nueva. Giros de 0°, 45°, 90° y 180°*

Qué hay que mirar: arriba a 90° los ojos quedan corridos y aparece una ceja de más; a 180° la cara es una mancha. Abajo, los cuatro giros salen bien.

3. La investigación: dónde está la falla

Lo primero es separar **detectar** (¿hay una cara?) de **ubicar** (¿dónde están sus ojos, nariz y boca?). Se dibujaron el recuadro (verde) y los 5 puntos clave (rojos) que devuelve el detector en cada giro:

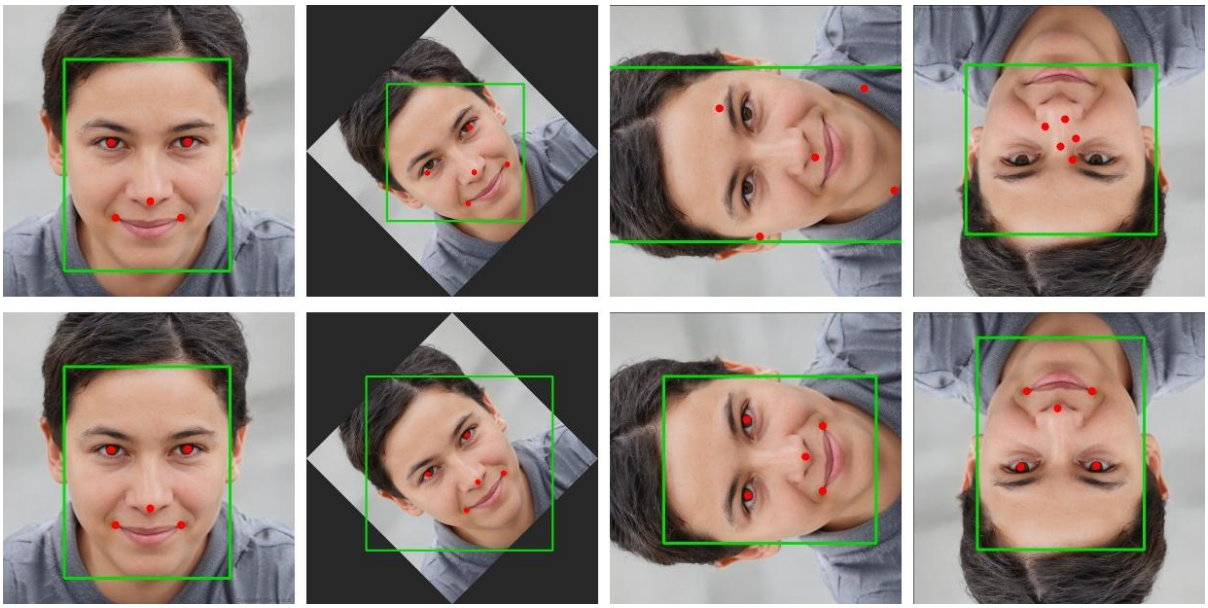


Figura 2. *Recuadro y 5 puntos clave. Arriba, versión anterior; abajo, versión nueva*

1. **La cara sí se detecta** en los cuatro giros, también en la versión anterior (puntuación entre 0,75 y 0,86).
2. **Los puntos están mal** a 90° y 180° en la versión anterior: a 90° el recuadro ocupa media imagen y los puntos caen en la frente y el hombro; a 180° se amontonan sobre la nariz.
3. Como inswapper recorta y orienta la cara **usando esos 5 puntos**, si los puntos están mal el recorte está mal y el resultado se deforma.

Causa: el detector de buffalo_l fue entrenado sobre todo con caras derechas. Reconoce "algo con forma de cara" girado, pero no sabe ubicar sus partes. A 45° todavía acierta; a 90° o más, no.

> **Para aprender:** Un modelo funciona bien con datos parecidos a los de su entrenamiento. Si no puedes cambiar el modelo, **cambia los datos** para que se parezcan: aquí, enderezar la imagen antes de mirarla.

4. La solución

Había varias opciones:

Opción	Ventaja	Desventaja
Usar un detector entrenado con caras giradas	Una sola pasada	Otro modelo que descargar e integrar
Girar siempre la imagen en los 4 ángulos	Simple	4 veces más lento en todos los fotogramas
Girar solo si hace falta y enderezar las inclinadas (elegida)	Igual de rápido con caras derechas	Más código

4.1 Detectar probando giros

`detectar_con_rotacion` prueba 0°, 90°, 180° y 270°, **en ese orden**, y para en cuanto encuentra caras confiables:

```
ANGULOS_DETECCION = (0, 90, 180, 270)

def detectar_con_rotacion(self, frame, angulos=None):
    candidatas = []
    for ang in angulos or ANGULOS_DETECCION:
        rot, M_inv = (frame, None) if ang == 0 else self._rotar(frame, ang)
        caras = Analyzer_Faces._ANALIZADOR_CARA.get(rot)
        confiables = all(abs(self.inclinacion(c)) <= 45 for c in caras)
        if M_inv is not None:
            for c in caras:
                self._mapear_cara(c, M_inv)     # coordenadas de vuelta al frame original
        candidatas.extend(caras)
        if len(caras) > 0 and confiables:
            break                               # con caras derechas: una sola pasada
    caras = self._unir_duplicadas(candidatas)
    ...
```

- **Confiable** significa que la línea de los ojos queda casi horizontal (±45°) en ese giro. Así se descartan detecciones como la de 90° de la figura 2.
- `_rotar` gira la imagen **agrandando el lienzo** para no cortar las esquinas, y devuelve la matriz inversa `M_inv`.
- `_mapear_cara` aplica `M_inv` al recuadro, a los 5 puntos y a los 106 puntos finos: la cara queda descrita en el frame **original**, que es donde se va a pegar.
- `_unir_duplicadas`: si la misma cara apareció en dos giros (centros a menos de medio tamaño de cara), se queda con la de mayor puntuación.

4.2 Enderezar las caras inclinadas

Una cara inclinada 30° o 60° no necesita un giro de 90°, pero sus puntos tampoco son precisos. `_refinar_cara_inclinada` mide la inclinación con la línea de los ojos y, si pasa de 30°, gira la imagen justo ese ángulo y vuelve a detectar:

```
def inclinacion(self, cara):
    vx, vy = cara.kps[1] - cara.kps[0]          # del ojo izquierdo al derecho
    return float(np.degrees(np.arctan2(vy, vx)))  # 0 = ojos horizontales
```

Solo acepta la nueva detección si es **la misma cara** (su centro está a menos de la distancia entre ojos) y su puntuación no baja más de un 10 %. Si no, se queda con la original: el refinado nunca empeora el resultado.

4.3 La máscara de oclusión alineada

XSeg (el modelo de oclusión) también espera caras derechas. Antes recibía un **rectángulo** recortado alrededor de la cara; con la cara girada, el modelo veía una cara de costado y la máscara salía mal.

Ahora `align_face` usa los 5 puntos para llevar la cara a una **posición estándar** (la plantilla FFHQ: dónde están los ojos, la nariz y la boca en una foto de carnet de 256×256). Se calcula la máscara sobre esa cara derecha y se devuelve a su lugar con la matriz inversa:

```
dst = ((FFHQ_TEMPLATE - 0.5) * margen + 0.5) * size          # puntos destino en 256x256
M, _ = cv2.estimateAffinePartial2D(face.kps, dst, method=cv2.RANSAC)  # giro + escala + traslado
crop = cv2.warpAffine(frame, M, (size, size), borderMode=cv2.BORDER_REPLICATE)
...                                                           # XSeg sobre la cara derecha
full = cv2.warpAffine(mask, cv2.invertAffineTransform(M), (w, h))   # máscara de vuelta
```

El interruptor `USAR_ALINEACION = True` permite volver al método anterior si algún día hiciera falta comparar.

> **Para aprender:** Este es el mismo patrón en las tres piezas: **llevar a una posición estándar → procesar → devolver con la matriz inversa**. Lo usan casi todos los programas de rostros (inswapper, GPEN, XSeg).

4.4 La corrección de los índices

Al revisar `Face_Swaper.py` apareció un error de nombre de variable:

Antes	Después
`for i in range(min(len(caras_src), len(caras_dst), len(index_src), len(indices_dst))):`	`for i in range(min(len(caras_src), len(caras_dst), len(indices_src), len(indices_dst))):`

`index_src` es el **texto** que escribió el usuario (`"0,1"`, de 3 caracteres) e `indices_src` es la **lista** `[0, 1]` (2 elementos). Con el texto, el bucle podía intentar usar un tercer índice que no existe.

Se reprodujo con una fuente de 3 caras (índices `"0,1"`) y un destino de 3 caras (índices `"0,1,2"`):

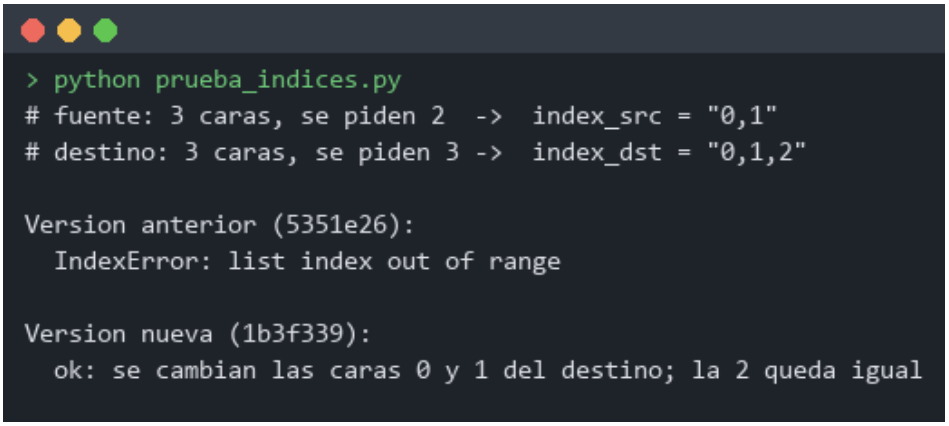


Figura 3. *La misma prueba con las dos versiones*



Figura 4. *Fuente (arriba), destino (centro) y resultado de la versión nueva (abajo): se cambian las caras 0 y 1; la 2 queda igual*

5. Cómo se comprobó

Prueba	Versión anterior	Versión nueva
Cara derecha (0°)	Bien	Bien, igual
Inclinada 45°	Bien	Bien
De costado (90°)	Puntos mal, cara deformada	Bien
Boca abajo (180°)	Puntos mal, mancha	Bien
Índices "0,1" → "0,1,2"	`IndexError`	Bien
Tiempo por imagen girada (CPU)	≈3,3 s	≈5,6–6,4 s

El tiempo sube con las caras giradas porque se detecta más de una vez (giros y refinado). Con caras derechas, que son la mayoría en un video normal, no hay pasadas extra.

> **Nota:** Solo se probó con una cara de IA en 4 giros y con imágenes, no con video. No se midió el ángulo exacto a partir del cual falla la versión anterior (entre 45° y 90°).

6. Cómo lo pruebas tú

1. Toma cualquier foto de prueba y gírala 90° con el visor de fotos de Windows (o usa un video donde alguien se acuesta).
2. Cámbiale la cara con TransferIa.

3. Verás la cara cambiada en su sitio, girada como la original.
4. En la versión actual puedes desactivarlo con `--sin-inclinadas` (CLI) o en la ventana, si quieres más velocidad y sabes que todas las caras están derechas.

7. Lo que aprendimos

- Que el detector diga "hay una cara" no significa que sepa dónde están sus partes: hay que mirar los puntos clave.
- Si el modelo no sabe trabajar con algo, se transforman los datos, se procesan y se deshace la transformación.
- Probar los casos caros solo cuando hacen falta mantiene la velocidad en el caso común.
- Un nombre parecido (`index_src` / `indices_src`) es una trampa: los nombres deben dejar claro si es texto o lista.

Glosario

Término	Significado
Puntos clave (kps)	Los 5 puntos de la cara: los dos ojos, la nariz y las dos comisuras de la boca.
det_score	Puntuación de confianza del detector, de 0 a 1.
Matriz afín	Una tabla de 2×3 números que describe un giro, un cambio de tamaño y un desplazamiento; su inversa los deshace.
Plantilla FFHQ	Posiciones estándar de ojos, nariz y boca en una cara alineada, usadas por muchos modelos de rostros.
RANSAC	Método que calcula la matriz ignorando puntos que no encajan (por ejemplo, un punto mal detectado).